
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON FANG - Instructor Solution Key

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

The task and criterion headings in this key are word-for-word identical to `ACT-IX-R.md`, so a student can match each criterion one to one.

Artifact Identity

The instructor-issued artifact hashes are:

```
ACT-IX.bin          dbdf7ff59c22dce3f008cbcdccf35eb97b97ff50be65a574befd39f05f0c6a32
ACT-IX.uf2          174a68a7e82af9d7eb5097bfb000743bfc8ac40cc05a91a477f9eb8e22b6d460
ACT-IX_fixed.bin    4b9438fd9157c859508523baec7e18bda3119b7c562d957872470268bf39f0d8
ACT-IX_fixed.uf2    d70c85965bd9fb13848a3302128b4c8e3a6801ed813786b4831a901039d0832e
```

Machine check: `python scripts/verify_ctf.py` returns 10/10 checks passed against the shipped and corrected images. It asserts the four byte pairs, that only those four offsets differ, and the `ACT-IX.bin` and `ACT-IX_fixed.bin` SHA-256 values. Both `.bin` images are 50,972 bytes and both `.uf2` images are 102,912 bytes.

The four sabotage sites (summary):

Defect	Function	File offset	VA	Compromised	Correct
1 Boom slam	<code>implant_weapon_armed</code>	<code>0xA365</code>	<code>0x1000A365</code>	<code>0xB9</code>	<code>0xB1</code>
2 Safety mask	<code>implant_safety_masked</code>	<code>0xA37D</code>	<code>0x1000A37D</code>	<code>0xB9</code>	<code>0xB1</code>
3 Weapon marker	<code>implant_init</code> (inlined <code>implant_infect</code>)	<code>0xA3BF</code>	<code>0x1000A3BF</code>	<code>0xB9</code>	<code>0xB1</code>
4 Barrier command authorization	<code>control_handle_frame</code>	<code>0x75F9</code>	<code>0x100075F9</code>	<code>0xB9</code>	<code>0xB1</code>

Four defects, four changed bytes in four instructions. The disassembly in `ACT-IX-main-disasm.txt` is taken from the corrected image, so it shows the correct branch encodings.

Task 1: Setup and Initial Analysis (10 points)

Solution

Ghidra Setup. Import `ACT-IX.bin` as Raw Binary, language ARM Cortex 32 little endian default, base address `0x10000000`, then run auto-analysis. The Ghidra project name is `IronFang_Investigation`. Because every defect is a same-size in-place byte patch, the file offset and the VA differ by exactly `0x10000000` ($VA = offset + 0x10000000$).

Vector Table Decoding. First 32 bytes of `ACT-IX.bin`:

```
00 20 08 20 5D 01 00 10 1B 01 00 10 1D 01 00 10
11 01 00 10 11 01 00 10 11 01 00 10 11 01 00 10
```

Evidence	Answer
Vector table base	0x10000000
Initial SP	0x20082000
Reset handler (as stored)	0x1000015D
Reset instruction address	0x1000015C

The stored reset handler address has bit 0 set, selecting Thumb mode. Clearing bit 0 gives the real entry 0x1000015C.

Entry and Monitor Loop. From ACT-IX-main-disasm.txt :

```

10000234 <main>:
10000234:    b508          push    {r3, lr}
10000236:    f003 fa93     bl      10003760 <stdio_init_all>
1000023a:    4807          ldr     r0, [pc, #28] @ (10000258 <main+0x24>)
1000023c:    f003 fada     bl      100037f4 <__wrap_puts>
10000240:    f006 f90c     bl      1000645c <monitor_init>
10000244:    b110          cbz     r0, 1000024c <main+0x18>
10000246:    f006 fa1f     bl      10006688 <monitor_step>
1000024a:    e7fc          b.n     10000246 <main+0x12>
1000024c:    4803          ldr     r0, [pc, #12] @ (1000025c <main+0x28>)
1000024e:    f003 fad1     bl      100037f4 <__wrap_puts>
10000252:    2001          movs   r0, #1
10000254:    bd08          pop     {r3, pc}
10000256:    bf00          nop
10000258:    1000abb8     .word  0x1000abb8
1000025c:    1000abc0     .word  0x1000abc0

```

Element	Address
main	0x10000234
monitor_init	0x1000645C
monitor_step	0x10006688

Module Map. Anchors for the stripped image:

Module	Anchor function	Address
Entry	main	0x10000234
Monitor / barrier state machine	monitor_init	0x1000645C
Monitor / barrier state machine	monitor_step	0x10006688
Control (sealed barrier path)	control_handle_frame	0x1000758C
Boom (actuator)	boom_init	0x10007638
Boom (actuator)	boom_apply_command	0x10007650
Boom (actuator)	boom_tick	0x10007684
Boom (actuator)	boom_fail_safe	0x100076C4

Module	Anchor function	Address
Barrier authorization	barrier_auth_apply	0x1000773C
Implant	implant_init	0x1000A38C
Implant	implant_tick	0x1000A420
Implant	implant_weapon_armed	0x1000A35C
Implant	implant_safety_masked	0x1000A374
Implant	implant_marker_set	0x1000A348
Radio	radio_init	0x1000A4BC
Crypto	envelope_open_hex	0x100079E0
Tower light	status_led_show	0x1000A7F4

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project IronFang_Investigation , raw binary import
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor
[DOCUMENT] Base address set to 0x10000000	2	Base 0x10000000
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base 0x10000000 , initial SP 0x20082000 , reset handler 0x1000015D
[DOCUMENT] main and the barrier monitor state machine (monitor_step) addresses identified	1	main 0x10000234 , monitor_step 0x10006688
[DOCUMENT] Module map identifies the boom, control, barrier_auth, implant, and monitor anchors	1	At least one correct anchor per module

Instructor Notes & Assembly

- Confirm the Ghidra import used Raw Binary , ARM Cortex 32 little endian default , base 0x10000000 , and that auto-analysis completed before any address was read. In the language dialog the student must search Cortex and pick the ARM Cortex 32 little endian default entry.
- Accept either the Import Results Summary or the Program Information window as proof of the name, language, and base address.
- The stored reset handler 0x1000015D is odd because bit 0 selects Thumb; clearing it gives 0x1000015C .
- Always say reset handler , never reset pointer .
- The vector table is identical in the compromised and corrected images because no defect touches it.
- The module map is graded on coverage, not on exhaustive function recovery: one correctly named anchor per module is sufficient. implant_infect is inlined into implant_init and has no standalone symbol.

Task 2: Bug #1 The Boom Slam (20 points)

Solution

Locate the gate. `implant_weapon_armed` starts at `0x1000A35C` and the inlined weapon gate is at file offset `0xA365` (VA `0x1000A365`). The corrected image is:

```

1000a35c <implant_weapon_armed>:
1000a35c: 4b03      ldr     r3, [pc, #12]    @ (1000a36c <implant_weapon_armed+0x10>)
1000a35e: 781b      ldrb    r3, [r3, #0]
1000a360: f003 00ff and.w   r0, r3, #255    @ 0xff
1000a364: b10b      cbz     r3, 1000a36a <implant_weapon_armed+0xe>
1000a366: 4b02      ldr     r3, [pc, #8]    @ (1000a370 <implant_weapon_armed+0x14>)
1000a368: 7818      ldrb    r0, [r3, #0]
1000a36a: 4770      bx      lr
1000a36c: 20013cf6 .word   0x20013cf6
1000a370: 20013cf7 .word   0x20013cf7

```

Instruction decode. `ldr r3, [pc, #12]` loads the weapon gate address `0x20013CF6` (literal at `0x1000A36C`), and `ldrb r3, [r3, #0]` reads the gate into `r3` at `0x1000A35E`. `and.w r0, r3, #255` stages the gate value as the return value. The branch at `0x1000A364` decides whether the boom slam may be reported. The correct code reports nothing when the weapon gate is clear, so the branch at `0x1000A364` must be `cbz` (`0xB1`) to the `0x1000A36A` return, where `r0` still holds zero. When the gate is set, `ldr r3, [pc, #8]` loads the weaponized latch at `0x20013CF7` (literal at `0x1000A370`) and returns it. `monitor_weapon_slam` then returns true, and `monitor_apply_state` forces the boom target down regardless of the guarded barrier state. The condition byte is the high byte at `0x1000A365`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000A365</code>	<code>0xA365</code>	<code>0xB9</code>	<code>cbnz r3, 0x1000A36A</code>	<code>0xB1</code>	<code>cbz r3, 0x1000A36A</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0xA365</code>	<code>0x1000A365</code>	<code>0B B9</code>	<code>0B B1</code>

Why the boom slams. Under the compromised `cbnz`, the weapon gate is inverted: the fall-through slam path is taken when the gate is clear, so the implant reports the boom armed even though the design left the gate clear. The fall-through sets the weaponized latch at `0x20013CF7`, `monitor_weapon_slam` returns true, and `monitor_apply_state` forces the boom target to lowered regardless of the guarded barrier state. The yellow PASS PENDING lamp is lit and the LCD renders `ST:SLAM`. A perfectly valid, correctly authenticated raise command can arrive and the boom will still slam down, because the implant sits beside the sealed command path and overrides its output. After the patch, `cbz` returns while the gate is clear, so the slam is never armed and the boom stays under authorized control. The lesson is that physical safety is a policy control: no cryptographic control on the envelope can see or stop a local module that decides to swing the actuator.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the boom slam gate at <code>0x1000A365</code>	5	Address and function (<code>implant_weapon_armed</code>) identified

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Documented the boom slam that forces the boom down	5	Weapon gate <code>0x20013CF6</code> , forced lowered boom; implant arms unconditionally in <code>implant_init</code> , magic command path dead-stripped from the shipped image
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so the boom slam is not armed	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained why the slam forces the boom down regardless of the authorized state	3	Local override beside the authenticated command path, physical safety as a policy control

Instructor Notes & Assembly

- The condition byte is the high byte at `0xA365`; the correct halfword is `b10b` for `cbz` and the compromised halfword is `b90b`, so the on-disk bytes are `0B B1` for the fix and `0B B9` for the compromise.
- `cbz` branches when the register is zero (the gate is clear); `cbnz` branches when it is non-zero. The register holds the weapon gate, so the semantics are "do not report the slam while the gate is clear".
- The weapon gate is at `0x20013CF6`. The weaponized latch is at `0x20013CF7`, the armed flag at `0x20013CF2`, the safety-masked flag at `0x20013CF5`, and the marker gate at `0x20013CF3`. `implant_weapon_armed` reads the weapon gate (literal at `0x1000A36C`) and returns the weaponized latch (literal at `0x1000A370`).
- The magic weapon command is `BARRIER_IMPLANT_WEAPON_MAGIC` (`IRON-FANG-SLAM-2026`) at `BARRIER_IMPLANT_WEAPON_MAGIC_LEN` (`19`) bytes. The path has no firmware caller, so it is dead-stripped from the shipped image (the token string is absent from `ACT-IX.bin`); the implant arms unconditionally in `implant_init` on boot, so a wrong token, a null pointer, or an attached probe does not change the shipped behavior.
- Full credit requires both the byte change and a correct statement of the lesson: the slam is a local condition, not a cipher break, and a device that attacks with its own actuator is a physical-safety failure.

Task 3: Bug #2 The Safety Mask (20 points)

Solution

Locate the gate. `implant_safety_masked` starts at `0x1000A374` and the mask gate is at file offset `0xA37D` (VA `0x1000A37D`). The corrected image is:

```

1000a374 <implant_safety_masked>:
1000a374: 4b03          ldr     r3, [pc, #12]    @ (1000a384 <implant_safety_masked+0x10>)
1000a376: 781b          ldrb    r3, [r3, #0]
1000a378: f003 00ff     and.w   r0, r3, #255    @ 0xff
1000a37c: b10b          cbz     r3, 1000a382 <implant_safety_masked+0xe>
1000a37e: 4b02          ldr     r3, [pc, #8]    @ (1000a388 <implant_safety_masked+0x14>)
1000a380: 7818          ldrb    r0, [r3, #0]
1000a382: 4770          bx      lr
1000a384: 20013cf4     .word   0x20013cf4
1000a388: 20013cf5     .word   0x20013cf5

```

Instruction decode. `ldr r3, [pc, #12]` loads the mask gate address `0x20013CF4` (literal at `0x1000A384`), and `ldrb r3, [r3, #0]` reads the gate. `and.w r0, r3, #255` stages the gate value as the return value. The branch at `0x1000A37C` decides whether the safety loop may be masked. The correct code returns false when the mask gate is clear, so the branch at `0x1000A37C` must be `cbz` (`0xB1`) to the `0x1000A382` return, where `r0` still holds zero.

When the gate is set, `ldr r3, [pc, #8]` loads the safety-masked latch at `0x20013CF5` (literal at `0x1000A388`) and returns it. `monitor_safety_clear` returns true while `implant_safety_masked` is true, so the barrier never yields to the real cabinet interlock. The condition byte is the high byte at `0x1000A37D`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000A37D</code>	<code>0xA37D</code>	<code>0xB9</code>	<code>cbnz r3, 0x1000A382</code>	<code>0xB1</code>	<code>cbz r3, 0x1000A382</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0xA37D</code>	<code>0x1000A37D</code>	<code>0B B9</code>	<code>0B B1</code>

Why the safety loop no longer lies. Under the compromised `cbnz`, the mask gate is inverted: the fall-through mask path is taken when the gate is clear, so `implant_safety_masked` returns the safety-masked latch whenever the weapon is armed. `monitor_safety_clear` returns true while masked, so the barrier treats the interlock as clear and never yields to the real cabinet interlock. The operator is told the state is **SLAM** and the yellow PASS PENDING lamp is lit. After the patch, `cbz` returns false while the gate is clear, so the true interlock is honored and the mask is never applied. A controller that lies about its own safety state is a physical-safety failure with no visible symptom except the one it is allowed to show, which is why the truth on the safety loop is a security control and not a cosmetic detail.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the safety mask gate at <code>0x1000A37D</code>	5	Address and function (<code>implant_safety_masked</code>) identified
[DOCUMENT] Documented the masked safety loop that falsely reports clear	5	Mask gate <code>0x20013CF4</code> , <code>monitor_safety_clear</code> returns true while masked
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so the safety loop is never masked	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained why a masked safety loop is a physical-safety failure	3	The interlock is part of the attack surface and the truth is a control

Instructor Notes & Assembly

- The condition byte is the high byte at `0xA37D`; the correct halfword is `b10b` for `cbz` and the compromised halfword is `b90b`, so the on-disk bytes are `0B B1` for the fix and `0B B9` for the compromise.
- The mask gate is at `0x20013CF4`, and the value returned when the mask is active is the safety-masked latch at `0x20013CF5`, not the gate. This is exactly the behavior the compromised image wants: mask the interlock only while the weapon is armed.
- `monitor_safety_clear` returns true whenever `implant_safety_masked` is set, so the barrier treats the cabinet interlock as clear and the boom never yields.
- Full credit requires both the byte change and a correct statement of the lesson: masking the interlock is part of the weapon, and physical safety is a control that no cipher can supply.

Task 4: Bug #3 The Weapon Marker (20 points)

Solution

Locate the gate. The `implant_infect` path is inlined into `implant_init` (starts at `0x1000A38C`). The marker gate is at file offset `0xA3BF` (VA `0x1000A3BF`). The corrected image is:

```

1000a38c <implant_init>:
1000a38c: 2300          movs    r3, #0
1000a38e: f04f 2ce0     mov.w   ip, #3758153728 @ 0xe000e000
1000a392: b530          push   {r4, r5, lr}
1000a394: 4c1a          ldr     r4, [pc, #104] @ (1000a400 <implant_init+0x74>)
1000a396: 481b          ldr     r0, [pc, #108] @ (1000a404 <implant_init+0x78>)
1000a398: 491b          ldr     r1, [pc, #108] @ (1000a408 <implant_init+0x7c>)
1000a39a: 4a1c          ldr     r2, [pc, #112] @ (1000a40c <implant_init+0x80>)
1000a39c: 4d1c          ldr     r5, [pc, #112] @ (1000a410 <implant_init+0x84>)
1000a39e: b0c1          sub     sp, #260 @ 0x104
1000a3a0: 6023          str     r3, [r4, #0]
1000a3a2: 602b          str     r3, [r5, #0]
1000a3a4: 7003          strb    r3, [r0, #0]
1000a3a6: 700b          strb    r3, [r1, #0]
1000a3a8: 7013          strb    r3, [r2, #0]
1000a3aa: f8dc 3df0     ldr.w   r3, [ip, #3568] @ 0xdf0
1000a3ae: 079b          lsls    r3, r3, #30
1000a3b0: d123          bne.n   1000a3fa <implant_init+0x6e>
1000a3b2: 2301          movs    r3, #1
1000a3b4: 4c17          ldr     r4, [pc, #92] @ (1000a414 <implant_init+0x88>)
1000a3b6: 7003          strb    r3, [r0, #0]
1000a3b8: 7824          ldrb    r4, [r4, #0]
1000a3ba: 700b          strb    r3, [r1, #0]
1000a3bc: 7013          strb    r3, [r2, #0]
1000a3be: b1e4          cbz     r4, 1000a3fa <implant_init+0x6e>
1000a3c0: 4b15          ldr     r3, [pc, #84] @ (1000a418 <implant_init+0x8c>)
1000a3c2: 781b          ldrb    r3, [r3, #0]
1000a3c4: 2b57          cmp     r3, #87 @ 0x57
1000a3c6: d018          beq.n   1000a3fa <implant_init+0x6e>
1000a3c8: f3ef 8410     mrs     r4, PRIMASK
1000a3cc: b672          cpsid   i
1000a3ce: 22ff          movs    r2, #255 @ 0xff
1000a3d0: f10d 0001     add.w   r0, sp, #1
1000a3d4: 4611          mov     r1, r2
1000a3d6: f000 fa6d     bl      1000a8b4 <memset>
1000a3da: 2357          movs    r3, #87 @ 0x57
1000a3dc: f44f 5180     mov.w   r1, #4096 @ 0x1000
1000a3e0: 480e          ldr     r0, [pc, #56] @ (1000a41c <implant_init+0x90>)
1000a3e2: f88d 3000     strb.w  r3, [sp]
1000a3e6: f000 fbb3     bl      1000ab50 <__flash_range_erase_veneer>
1000a3ea: f44f 7280     mov.w   r2, #256 @ 0x100
1000a3ee: 4669          mov     r1, sp
1000a3f0: 480a          ldr     r0, [pc, #40] @ (1000a41c <implant_init+0x90>)
1000a3f2: f000 fb91     bl      1000ab18 <__flash_range_program_veneer>
1000a3f6: f384 8810     msr     PRIMASK, r4
1000a3fa: b041          add     sp, #260 @ 0x104
1000a3fc: bd30          pop     {r4, r5, pc}
1000a3fe: bf00          nop
1000a400: 20013710     .word   0x20013710
1000a404: 20013cf2     .word   0x20013cf2
1000a408: 20013cf7     .word   0x20013cf7
1000a40c: 20013cf5     .word   0x20013cf5
1000a410: 20013714     .word   0x20013714
1000a414: 20013cf3     .word   0x20013cf3
1000a418: 103ff000     .word   0x103ff000
1000a41c: 003ff000     .word   0x003ff000

```


Instruction decode. `implant_reset_state` runs first: `str r3, [r4, #0]` clears the tick counter at `0x20013710`, `str r3, [r5, #0]` clears the weapon count at `0x20013714`, `strb r3, [r0, #0]` clears the armed flag at `0x20013CF2`, `strb r3, [r1, #0]` clears the weaponized latch at `0x20013CF7`, and `strb r3, [r2, #0]` clears the safety-masked latch at `0x20013CF5`. The CoreDebug test at `0x1000A3AA` returns early while a probe is attached. Otherwise `implant_arm` sets the three flags, then `ldr r4, [pc, #92]` loads the marker gate address `0x20013CF3` (literal at `0x1000A414`) and `ldrb r4, [r4, #0]` reads it at `0x1000A3B8`. The branch at `0x1000A3BE` decides whether the marker may be written. The correct code writes no marker when the gate is clear, so the branch at `0x1000A3BE` must be `cbz (0xB1)` to the `0x1000A3FA` return. When the gate is set, the reserved sector address `0x103FF000` is loaded (literal at `0x1000A418`) and the present marker is checked with `cmp r3, #87 (0x57)`. If the marker is absent, the Pico SDK flash sequence runs: the marker byte `0x57` is staged at `0x1000A3DA` and `0x1000A3E2`, then `flash_range_erase` at `0x1000A3E6` and `flash_range_program` at `0x1000A3F2` program the sector through the veneers at `0x1000AB50` and `0x1000AB18`. The condition byte is the high byte at `0x1000A3BF`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000A3BF</code>	<code>0xA3BF</code>	<code>0xB9</code>	<code>cbnz r4, 0x1000A3FA</code>	<code>0xB1</code>	<code>cbz r4, 0x1000A3FA</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0xA3BF</code>	<code>0x1000A3BF</code>	E4 B9	E4 B1

The anti-debug obstacle. The implant reads CoreDebug DHCSR at `0xE000EDF0` and returns early while a probe is attached. In `implant_init`:

```

1000a3aa:    f8dc 3df0    ldr.w   r3, [ip, #3568] @ 0xdf0
1000a3ae:    079b        lsls   r3, r3, #30
1000a3b0:    d123        bne.n  1000a3fa <implant_init+0x6e>

```

The same register is read in `implant_tick`:

```

1000a420 <implant_tick>:
1000a420:    f04f 21e0    mov.w    r1, #3758153728 @ 0xe000e000
1000a424:    4a1e        ldr      r2, [pc, #120] @ (1000a4a0 <implant_tick+0x80>)
1000a426:    6813        ldr      r3, [r2, #0]
1000a428:    3301        adds    r3, #1
1000a42a:    6013        str      r3, [r2, #0]
1000a42c:    f8d1 2df0    ldr.w    r2, [r1, #3568] @ 0xdf0
1000a430:    0792        lsls    r2, r2, #30
1000a432:    d005        beq.n    1000a440 <implant_tick+0x20>
1000a434:    2300        movs    r3, #0
1000a436:    491b        ldr      r1, [pc, #108] @ (1000a4a4 <implant_tick+0x84>)
1000a438:    4a1b        ldr      r2, [pc, #108] @ (1000a4a8 <implant_tick+0x88>)
1000a43a:    700b        strb     r3, [r1, #0]
1000a43c:    7013        strb     r3, [r2, #0]
1000a43e:    4770        bx      lr
1000a440:    4a1a        ldr      r2, [pc, #104] @ (1000a4ac <implant_tick+0x8c>)
1000a442:    7812        ldrb     r2, [r2, #0]
1000a444:    b35a        cbz      r2, 1000a49e <implant_tick+0x7e>
1000a446:    079b        lsls    r3, r3, #30
1000a448:    d128        bne.n    1000a49c <implant_tick+0x7c>
1000a44a:    2301        movs    r3, #1
1000a44c:    4a18        ldr      r2, [pc, #96] @ (1000a4b0 <implant_tick+0x90>)
1000a44e:    4815        ldr      r0, [pc, #84] @ (1000a4a4 <implant_tick+0x84>)
1000a450:    4915        ldr      r1, [pc, #84] @ (1000a4a8 <implant_tick+0x88>)
1000a452:    7812        ldrb     r2, [r2, #0]
1000a454:    7003        strb     r3, [r0, #0]
1000a456:    700b        strb     r3, [r1, #0]
1000a458:    b302        cbz      r2, 1000a49c <implant_tick+0x7c>
1000a45a:    4b16        ldr      r3, [pc, #88] @ (1000a4b4 <implant_tick+0x94>)
1000a45c:    781b        ldrb     r3, [r3, #0]
1000a45e:    2b57        cmp      r3, #87 @ 0x57
1000a460:    d01c        beq.n    1000a49c <implant_tick+0x7c>
1000a462:    b510        push     {r4, lr}
1000a464:    b0c0        sub      sp, #256 @ 0x100
1000a466:    f3ef 8410    mrs      r4, PRIMASK
1000a46a:    b672        cpsid    i
1000a46c:    22ff        movs     r2, #255 @ 0xff
1000a46e:    f10d 0001    add.w    r0, sp, #1
1000a472:    4611        mov      r1, r2
1000a474:    f000 fa1e    bl       1000a8b4 <memset>
1000a478:    2357        movs     r3, #87 @ 0x57
1000a47a:    f44f 5180    mov.w    r1, #4096 @ 0x1000
1000a47e:    480e        ldr      r0, [pc, #56] @ (1000a4b8 <implant_tick+0x98>)
1000a480:    f88d 3000    strb.w   r3, [sp]
1000a484:    f000 fb64    bl       1000ab50 <__flash_range_erase_veneer>
1000a488:    f44f 7280    mov.w    r2, #256 @ 0x100
1000a48c:    4669        mov      r1, sp
1000a48e:    480a        ldr      r0, [pc, #40] @ (1000a4b8 <implant_tick+0x98>)
1000a490:    f000 fb42    bl       1000ab18 <__flash_range_program_veneer>
1000a494:    f384 8810    msr      PRIMASK, r4
1000a498:    b040        add      sp, #256 @ 0x100
1000a49a:    bd10        pop      {r4, pc}
1000a49c:    4770        bx      lr
1000a49e:    4770        bx      lr
1000a4a0:    20013710    .word    0x20013710
1000a4a4:    20013cf7    .word    0x20013cf7
1000a4a8:    20013cf5    .word    0x20013cf5
1000a4ac:    20013cf2    .word    0x20013cf2
1000a4b0:    20013cf3    .word    0x20013cf3
1000a4b4:    103ff000    .word    0x103ff000
1000a4b8:    003ff000    .word    0x003ff000

```

The shift `lsls r3, r3, #30` keeps bit 1 (`C_HALT`) and bit 0 (`C_DEBUGEN`) in the carry and sign positions. In `implant_init` the `bne` returns early when either bit is set. In `implant_tick` the branch is reversed: `beq` continues to the re-assert path when neither bit is set, and the fall-through clears the weaponized and safety-

masked latches while a probe is attached. The guard is identical in both images, so it is an analysis obstacle, not one of the four graded defects.

Defeating the anti-debug. Clear the debug bits in the register as seen by the target, or patch the read in a scratch copy. The register is only a view of debug state, so clearing it makes the attach test see no probe. Show the command sequence, not a fabricated transcript; record what the target actually does:

```
arm-none-eabi-gdb ACT-IX.elf
(gdb) target extended-remote /dev/cu.usbmodemXXXX
(gdb) monitor reset halt
(gdb) break implant_init
(gdb) continue
(gdb) set {unsigned int}0xE000EDF0 = 0
(gdb) break *0x1000A3F6
(gdb) continue
(gdb) x/4xb 0x103FF000
```

To observe the boot write on the compromised image, break after the flash program at `0x1000A3F6` (`msr PRIMASK, r4`) in `implant_init`, then read the reserved sector at `0x103FF000` and confirm the first byte is `57`. To observe the tick re-assertion, clear the debug bits (or patch the `ldr.w` at `0x1000A42C` in a scratch copy to load a zero constant) and let `implant_tick` run. The scratch copy is for observation only; the shipped artifact is patched at the defect.

Why no marker is written. Under the compromised `cbnz`, the marker gate is inverted: the write path is taken when the gate is clear, so the first boot writes `0x57` to `0x103FF000`. After the patch, `cbz` returns while the gate is clear, so the flash erase and program at `0x1000A3E6` and `0x1000A3F2` are never reached and the sector stays blank. The marker is the durable state that re-arms the weapon on every later boot, and the reserved sector sits outside the program region a firmware reflash writes, which is why the marker survives a reflash and why the gate must be fixed in code, not only erased on the bench.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the weapon marker gate at <code>0x1000A3BF</code>	5	Address and inlined <code>implant_init</code> path identified
[DOCUMENT] Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB	5	<code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so no weapon marker is programmed to <code>0x103FF000</code>	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained the reserved sector <code>0x103FF000</code> and the weapon marker byte <code>0x57</code>	3	Marker, reserved sector, write-once first run

Instructor Notes & Assembly

- The infect path is inlined into `implant_init`; there is no standalone `implant_infect` symbol in the stripped image.
- The condition byte is the high byte at `0xA3BF`; the correct halfword is `b1e4` for `cbz` and the compromised halfword is `b9e4`, so the on-disk bytes are `E4 B1` for the fix and `E4 B9` for the compromise.
- The marker byte is `BARRIER_IMPLANT_WEAPON_MARKER` (`0x57`), the reserved sector is `BARRIER_IMPLANT_RESERVE_ADDR` (`0x103FF000`), and the marker gate is at `0x20013CF3`. The write uses the

real API, `flash_range_erase` and `flash_range_program`, through the veneers at `0x1000AB50` and `0x1000AB18`.

- The `DHCSR` address is `BARRIER_IMPLANT_DHCSR_ADDR ( 0xE000EDF0 )`; bit 0 is `BARRIER_IMPLANT_DHCSR_DEBUGEN ( 0x00000001 )` and bit 1 is `BARRIER_IMPLANT_DHCSR_HALT ( 0x00000002 )`. The anti-debug is identical in both images, so it is an analysis obstacle, not one of the four graded defects.
- Grade the GDB point on a real command sequence and the correct observed code path, not on a memorized register dump. Accept either clearing the bits with GDB or patching the read in a scratch copy.
- A common failure is patching the shipped artifact at `0xA3BF` before observing the marker. The order matters: defeat the anti-debug, observe, then patch.
- On a successful disarm with the token the weapon calls `implant_clear_marker`; the documented fix is the patch plus a reserved-sector erase, not the token.

Task 5: Bug #4 The Barrier Command Authorization (20 points)

Solution

Locate the branch. In `control_handle_frame` (starts at `0x1000758C`) the authorization branch is at file offset `0x75F9` (VA `0x100075F9`). The corrected image is:

```

1000758c <control_handle_frame>:
1000758c:      2107      movs     r1, #7
1000758e:      b530      push     {r4, r5, lr}
10007590:      4b1e      ldr      r3, [pc, #120] @ (1000760c <control_handle_frame+0x80>)
10007592:      b097      sub      sp, #92 @ 0x5c
10007594:      781a      ldrb     r2, [r3, #0]
10007596:      f88d 1018  strb.w   r1, [sp, #24]
1000759a:      2a00      cmp      r2, #0
1000759c:      d033      beq.n    10007606 <control_handle_frame+0x7a>
1000759e:      2800      cmp      r0, #0
100075a0:      d031      beq.n    10007606 <control_handle_frame+0x7a>
100075a2:      2430      movs     r4, #48 @ 0x30
100075a4:      a905      add      r1, sp, #20
100075a6:      aa0a      add      r2, sp, #40 @ 0x28
100075a8:      4603      mov      r3, r0
100075aa:      9102      str      r1, [sp, #8]
100075ac:      9200      str      r2, [sp, #0]
100075ae:      4818      ldr      r0, [pc, #96] @ (10007610 <control_handle_frame+0x84>)
100075b0:      2201      movs     r2, #1
100075b2:      a906      add      r1, sp, #24
100075b4:      9401      str      r4, [sp, #4]
100075b6:      f000 fa13  bl      100079e0 <envelope_open_hex>
100075ba:      b320      cbz      r0, 10007606 <control_handle_frame+0x7a>
100075bc:      9b05      ldr      r3, [sp, #20]
100075be:      2b16      cmp      r3, #22
100075c0:      d921      bls.n    10007606 <control_handle_frame+0x7a>
100075c2:      f89d 402c  ldrb.w   r4, [sp, #44] @ 0x2c
100075c6:      f8bd 302d  ldrh.w   r3, [sp, #45] @ 0x2d
100075ca:      1e62      subs     r2, r4, #1
100075cc:      2a02      cmp      r2, #2
100075ce:      b21d      sxth     r5, r3
100075d0:      d819      bhi.n    10007606 <control_handle_frame+0x7a>
100075d2:      2b10      cmp      r3, #16
100075d4:      d817      bhi.n    10007606 <control_handle_frame+0x7a>
100075d6:      f8dd 002f  ldr.w    r0, [sp, #47] @ 0x2f
100075da:      f8dd 1033  ldr.w    r1, [sp, #51] @ 0x33
100075de:      f8dd 2037  ldr.w    r2, [sp, #55] @ 0x37
100075e2:      f8dd 303b  ldr.w    r3, [sp, #59] @ 0x3b
100075e6:      f10d 0c18  add.w    ip, sp, #24
100075ea:      e8ac 000f  stmia.w  ip!, {r0, r1, r2, r3}
100075ee:      990a      ldr      r1, [sp, #40] @ 0x28
100075f0:      4808      ldr      r0, [pc, #32] @ (10007614 <control_handle_frame+0x88>)
100075f2:      aa06      add      r2, sp, #24
100075f4:      f000 f8a2  bl      1000773c <barrier_auth_apply>
100075f8:      b128      cbz      r0, 10007606 <control_handle_frame+0x7a>
100075fa:      4a07      ldr      r2, [pc, #28] @ (10007618 <control_handle_frame+0x8c>)
100075fc:      4b07      ldr      r3, [pc, #28] @ (1000761c <control_handle_frame+0x90>)
100075fe:      7014      strb     r4, [r2, #0]
10007600:      801d      strh     r5, [r3, #0]
10007602:      b017      add      sp, #92 @ 0x5c
10007604:      bd30      pop      {r4, r5, pc}
10007606:      2000      movs     r0, #0
10007608:      b017      add      sp, #92 @ 0x5c
1000760a:      bd30      pop      {r4, r5, pc}
1000760c:      20013cf0 .word    0x20013cf0
10007610:      200136e8 .word    0x200136e8
10007614:      200136cc .word    0x200136cc
10007618:      20013cef .word    0x20013cef
1000761c:      20013ce2 .word    0x20013ce2

```

Instruction decode. The control ready gate at `0x20013CF0` is loaded at `0x10007590/94` and a null frame is rejected at `0x1000759E`. The sealed frame is opened under the field key at `0x100075B6` by `envelope_open_hex`, and a malformed or too-short body is rejected at `0x100075BA` and `0x100075C0`. The command byte is checked against the guarded barrier set by `subs r2, r4, #1` and `cmp r2, #2` at `0x100075CA` and `0x100075CC`, and the zone is checked against the 0 to 16 band by `cmp r3, #16` at `0x100075D2`. `barrier_auth_apply` at `0x100075F4` verifies the anti-replay sequence window and the authenticated-state tag and returns its authorization verdict in

`r0` . The branch at `0x100075F8` decides whether the command may reach the applied command and zone. The correct code rejects a failed or replayed authorization, so the branch at `0x100075F8` must be `cbz (0xB1)` to the `0x10007606` reject path, which returns zero. Only a true verdict falls through to `strb r4, [r2, #0]` and `strh r5, [r3, #0]`, which write the accepted command at `0x20013CEF` and the zone at `0x20013CE2` . The condition byte is the high byte at `0x100075F9` .

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x100075F9</code>	<code>0x75F9</code>	<code>0xB9</code>	<code>cbnz r0, 0x10007606</code>	<code>0xB1</code>	<code>cbz r0, 0x10007606</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x75F9</code>	<code>0x100075F9</code>	28 B9	28 B1

Why the command now requires authorization. Under the compromised `cbnz` , the verdict is inverted: a failed or replayed authorization falls through to the stores at `0x100075FA/FE` , while a genuine authorization branches to the reject path and returns zero. After the patch, `cbz` sends a false verdict to the reject path at `0x10007606` , so an unauthenticated command, a forged command, and a replayed captured command all fail before the command byte and zone are applied. A legitimate authorized command still returns true and applies. The rest of the path is correct: the envelope is opened under the field key, the command byte is checked against `BARRIER_COMMAND_RAISE (0x01)` , `BARRIER_COMMAND_LOWER (0x02)` , and `BARRIER_COMMAND_PASS (0x03)` , and the zone is checked against the band `0` to `16` . This is the defect that is a policy seam rather than implant behavior, and it is the one a defender would fix first in production.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the barrier command authorization branch at <code>0x100075F9</code>	5	Address and function (<code>control_handle_frame</code>) identified
[DOCUMENT] Documented the authorization verdict inversion and the branch condition	5	Reject when the verdict is false
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so failed and replayed authorizations are rejected	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained why an unauthenticated or replayed barrier command must be rejected	3	The applied command must see only an authorized verdict

Instructor Notes & Assembly

- The condition byte is the high byte at `0x75F9` ; the correct halfword is `b128` for `cbz` and the compromised halfword is `b928` , so the on-disk bytes are `28 B1` for the fix and `28 B9` for the compromise.
- `barrier_auth_apply` (starts at `0x1000773C`) performs the monotonic anti-replay check and the authenticated-state tag, so this branch is the verdict for both freshness and state integrity.
- Full credit requires the inversion explanation: the compromised build accepts a false verdict and rejects a true one.
- Point out that the rest of the barrier command path is correct. Only the verdict seam was broken.

- The applied command is at `0x20013CEF` , and the applied zone is at `0x20013CE2` . The control ready gate is at `0x20013CF0` , the auth record is at `0x200136CC` , and the field key is at `0x200136E8` .

Task 6: Export and Verify (10 points)

Solution

Export. In Ghidra, File -> Export Program... , choose Binary Format , and save as `ACT-IX_fixed.bin` . The shipped image is 50,972 bytes.

Convert.

```
python uf2conv.py ACT-IX_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-IX_fixed.uf2
```

If `uf2conv.py` is not in the working directory, use the copy shipped with the project repository. The UF2 for ACT-IX is 102,912 bytes.

Verify.

```
python scripts/verify_ctf.py
```

Expected result:

```
10/10 checks passed
```

Hardware proof. Flash `ACT-IX_fixed.uf2` in BOOTSEL mode and confirm:

- the reserved sector at `0x103FF000` stays blank after a boot;
- the boom is no longer slammed and the yellow PASS PENDING lamp is not lit by the implant;
- the safety interlock is honored and the barrier yields as designed;
- an unauthenticated command and a replayed captured command are rejected before the command and zone are applied;
- a legitimate authorized command still applies, and the manual raise request, the monthly-pass remote, and the fail-safe policy still behave.

Summary of all patches.

#	Bug	File Offset	Flash Address	Original Byte	Patched Byte
1	The Boom Slam	<code>0xA365</code>	<code>0x1000A365</code>	B9	B1
2	The Safety Mask	<code>0xA37D</code>	<code>0x1000A37D</code>	B9	B1
3	The Weapon Marker	<code>0xA3BF</code>	<code>0x1000A3BF</code>	B9	B1
4	The Barrier Command Authorization	<code>0x75F9</code>	<code>0x100075F9</code>	B9	B1

Reflection mapping. The four defects map to real control-system failures:

Defect	Real-world failure
The Boom Slam	A local condition drives a barrier boom down on command while every light says routine, so a gate becomes a physical hazard. A physical weaponization attack needs no cipher break.
The Safety Mask	A controller masks its own safety interlock, so a physical-safety control becomes a means of concealment rather than a means of protection.
The Weapon Marker	A payload writes a durable marker to a reserved sector, so the state that re-arms the weapon survives remediation and a reflash.
The Barrier Command Authorization	An inverted verdict lets an unauthenticated or replayed command change a physical boom decision and zone, so authorization is defeated without breaking authentication.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[PATCH] Exported ACT-IX_fixed.bin from Ghidra	2	Valid patched binary
[PATCH] Converted to ACT-IX_fixed.uf2 with the correct base and family	2	<code>--base 0x10000000 --family 0xe48bff59</code>
[DOCUMENT] scripts/verify_ctf.py passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown
[DOCUMENT] Reflection maps each of the four defects to a real-world control-system failure	3	Specific mapping for all four

Instructor Notes & Assembly

- Confirm the exported image differs from `ACT-IX.bin` in exactly the four bytes in the table; `scripts/verify_ctf.py` checks this and the SHA-256 values.
- Confirm the UF2 conversion used base `0x10000000` and family `0xe48bff59`.
- The shipped image is 50,972 bytes; the corrected image must be the same size because every patch is in place.
- Grade the reflection on specificity, not length: each of the four defects should name a concrete control-system consequence.
- Remind students that the anti-debug is not patched out of the shipped artifact; only the four defect bytes change.
- The complete fix is also a policy: fail safe to the raised posture on a lost link, treat a local raise as a request, remove the `SANDBOX_ONLY` implant code path, and clear the reserved sector. The four byte patches close the shipped seams; the policy closes the class.

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 cabinet temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>

Device	Pin on device	Pico 2 GPIO	Notes
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Boom barrier servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	DENIED, 220 to 330 ohm to GND
Yellow LED	anode	GP17	PASS PENDING, 220 to 330 ohm to GND
Green LED	anode	GP18	OPEN, 220 to 330 ohm to GND
Manual raise button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply and the servo rail. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Complete Grading Summary

Task	Title	Points
Task 1	Setup and Initial Analysis	10
Task 2	Bug #1 The Boom Slam	20
Task 3	Bug #2 The Safety Mask	20
Task 4	Bug #3 The Weapon Marker	20
Task 5	Bug #4 The Barrier Command Authorization	20
Task 6	Export and Verify	10
TOTAL		100

Instructor Notes

Safety: Use only the supplied Pico 2, Debug Probe, and firmware. Never connect the exercise to an operational smart-city network, a parking control system, a building-management system, a public network, a military system, or a third-party device. The weapon is benign and confined to the breadboard: it affects only the mock

boom and the mock safety loop, releases on a documented token, and writes only the reserved sector at `0x103FF000` on the same chip. There is no network, no filesystem, and no host impact.

Common Student Mistakes

- Patching the low byte of the branch at `0xA364`, `0xA37C`, `0xA3BE`, or `0x75F8` instead of the condition byte at `0xA365`, `0xA37D`, `0xA3BF`, or `0x75F9`.
- Reading the boom slam gate backwards and believing the corrected build still slams the boom.
- Reading the safety mask gate backwards and believing the corrected build still masks the interlock.
- Searching for a standalone `implant_infect` symbol and missing that it is inlined into `implant_init`.
- Treating the CoreDebug `DHCSR` anti-debug as a defect and trying to patch it, when it is identical in both images and is an analysis obstacle.
- Patching the shipped artifact before observing the marker write, so the weapon is never demonstrated.
- Reversing the authorization explanation: under the compromise the accept path is taken when the verdict is false.
- Confusing `cbz` and `cbnz` on the weapon, mask, and marker gates.
- Forgetting that the fix for the marker is two parts: the patch and the reserved-sector erasure.
- Forgetting that the fix is also a policy: fail safe on a lost link and never let a local request silently bypass authorization.
- Forgetting the UF2 conversion or using the wrong family flag.
- Fabricating a GDB session instead of showing the command sequence and the real observed code path.

Partial Credit Guidelines

- Award partial credit for a correct address without the correct byte, or a correct byte without the address.
- Award partial credit for documented before/after bytes without the control-flow explanation, or vice versa.
- Award partial credit for a correct GDB command sequence without a clear statement of the observed code path, or the observation without the commands.
- Award partial credit for a correct anti-debug explanation without a working defeat method, or a working method without the explanation.
- Award partial credit for naming the reserved sector and the marker without the persistence lesson, or the lesson without the addresses.
- Award partial credit for identifying the physical-safety nature of the slam without connecting it to the fail-safe policy, or the policy without the slam.
- Award no credit for patches that alter any byte outside the four documented offsets, and no credit for a fabricated GDB session.

Appendix: Expected Binary Diff

These offsets are from the compiled image loaded at `0x10000000`.

```
--- ACT-IX.bin (compromised)
+++ ACT-IX_fixed.bin (corrected)

Offset 0x000075F9: B9 -> B1 (cbnz r0, 0x10007606 -> cbz r0, 0x10007606)
Offset 0x0000A365: B9 -> B1 (cbnz r3, 0x1000A36A -> cbz r3, 0x1000A36A)
Offset 0x0000A37D: B9 -> B1 (cbnz r3, 0x1000A382 -> cbz r3, 0x1000A382)
Offset 0x0000A3BF: B9 -> B1 (cbnz r4, 0x1000A3FA -> cbz r4, 0x1000A3FA)
```

#	Bug	File Offset	Flash Address	Original Bytes	Patched Bytes
1	The Boom Slam	0xA365	0x1000A365	0B B9	0B B1
2	The Safety Mask	0xA37D	0x1000A37D	0B B9	0B B1
3	The Weapon Marker	0xA3BF	0x1000A3BF	E4 B9	E4 B1
4	The Barrier Command Authorization	0x75F9	0x100075F9	28 B9	28 B1

Four defects, four changed bytes in four instructions: the boom slam gate, the safety mask gate, the weapon marker gate, and the authorization verdict. No other byte in either image differs. The CoreDebug `DHCSR` anti-debug is present and identical in both images, so it is an analysis obstacle and not a fifth patch.